Unit 4 · Repeating actions in Python

CONCEPT & REASONING

A loop repeats a block of statements. This unit follows decisions (Unit 3) and prepares you for collections and functions (Unit 5).

EXAMPLE / TAKEAWAY

Learning outcomes: for and while; range; counters and totals; input validation; tracing and testing five days of expenditure.

One action versus repeated actions

CONCEPT & REASONING

A sequence carries out instructions once. A loop runs a body again according to a counting sequence or a condition.

EXAMPLE / TAKEAWAY

Five identical daily-input lines can become one input line inside a loop. This reduces duplication and makes changes easier.

An iteration is one pass

CONCEPT & REASONING

Every execution of a loop body is called an iteration. Values can change between iterations.

EXAMPLE / TAKEAWAY

Three passes might process Monday, Tuesday and Wednesday. The loop ends when there are no more values or the condition becomes false.

The anatomy of a for loop

CONCEPT & REASONING

The heading supplies successive values; the indented body runs for each value. The following unindented statement runs only once after the loop.

PYTHON EXAMPLE

```
for day in range(1, 4):
    print("Day", day)
print("Finished")
```

Read the output

CONCEPT & REASONING

For each value 1, 2 and 3, Python executes the body. Once the values are exhausted, execution continues below the loop.

PYTHON EXAMPLE

```
Day 1
Day 2
Day 3
Finished
```

range(3) starts at zero

CONCEPT & REASONING

One-argument range(n) yields integers starting at 0 and stopping before n. There are n iterations when n is non-negative.

PYTHON EXAMPLE

```
for number in range(3):
    print(number)
# 0, 1, 2
```

range(start, stop)

CONCEPT & REASONING

The start is included and the stop is excluded. Use stop one greater than the final included integer when counting upwards by one.

PYTHON EXAMPLE

```
for day in range(1, 6):
    print(day)
# 1, 2, 3, 4, 5
```

range(start, stop, step)

CONCEPT & REASONING

Step controls the size and direction of change. It must not be zero. Count backwards with a negative step.

PYTHON EXAMPLE

```
for n in range(2, 11, 2):
    print(n)
# 2, 4, 6, 8, 10
```

A countdown

CONCEPT & REASONING

Use a negative step with a start higher than the stop. Stop remains excluded.

PYTHON EXAMPLE

```
for remaining in range(5, 0, -1):
    print(remaining)
# 5, 4, 3, 2, 1
```

When a range is empty

CONCEPT & REASONING

If the arguments do not move towards the stop, no iterations occur. This is not an error.

PYTHON EXAMPLE

```
for n in range(3, 3):
    print("This never prints")
print("Done")
```

Indentation defines the body

CONCEPT & REASONING

Every statement at the same indentation level inside the loop repeats. A statement aligned with for is outside the body.

PYTHON EXAMPLE

```
for day in range(1, 4):
    print("Day", day)
    print("Recorded")
print("One summary")
```

Trace a for loop

CONCEPT & REASONING

Write the loop variable, calculations and output on each pass. A trace table makes hidden changes visible.

PYTHON EXAMPLE

```
for count in range(1, 4):
    square = count * count
    print(count, square)
# 1 1 / 2 4 / 3 9
```

A counter counts events

CONCEPT & REASONING

Set a counter to zero before the loop; add one when an event happens. A counter need not equal the loop variable.

PYTHON EXAMPLE

```
zero_days = 0
for day in range(1, 4):
    amount = float(input("Expense: "))
    if amount == 0:
        zero_days += 1
print(zero_days)  # 2 for inputs 0, 1000, 0
```

An accumulator adds values

CONCEPT & REASONING

Set a running total before the loop and add each new amount. Keeping total inside the loop without resetting it preserves earlier amounts.

PYTHON EXAMPLE

```
total = 0
for day in range(1, 4):
    amount = float(input("Expense: "))
    total += amount
print(total)  # 4000 for 2000, 1500, 500
```

Follow the running total

CONCEPT & REASONING

An accumulator changes after each input, allowing a final summary. Trace both the input and updated total.

EXAMPLE / TAKEAWAY

Beginning total 0

+ 2000 → 2000

+ 1500 → 3500

+ 500 → 4000

Input can occur inside a loop

CONCEPT & REASONING

input() returns text. Convert to float or int when arithmetic is needed. An f-string can show the current day number.

PYTHON EXAMPLE

```
for day in range(1, 4):
    amount = float(input(f"Day {day}: "))
    print("Entered:", amount)
```

A repeated expenditure example

CONCEPT & REASONING

A three-day total needs one accumulator and three numeric inputs; the summary is printed once outside the loop.

PYTHON EXAMPLE

```
total = 0.0
for day in range(1, 4):
    amount = float(input(f"Day {day}: "))
    total += amount
print(f"TZS {total:.2f}")
```

Average follows the total

CONCEPT & REASONING

Average is the sum divided by the number of entries. Divide after the loop by a known positive count.

PYTHON EXAMPLE

```
total = 15000
days = 5
average = total / days
print(f"TZS {average:.2f}")
# TZS 3000.00
```

Introducing while

CONCEPT & REASONING

while checks a Boolean condition before each iteration. Its body repeats as long as that condition is True.

PYTHON EXAMPLE

```
count = 1
while count <= 3:
    print(count)
    count += 1
# 1, 2, 3
```

Trace the while condition

CONCEPT & REASONING

The condition must eventually become false, unless a deliberate exit is used. Count moves from 1 through 4.

EXAMPLE / TAKEAWAY

count 1: 1 <= 3 → True

count 2: 2 <= 3 → True

count 3: 3 <= 3 → True

count 4: 4 <= 3 → False; stop

Why infinite loops happen

CONCEPT & REASONING

A while loop that never changes the condition variable may never finish. Look for progress towards the stopping condition.

PYTHON EXAMPLE

```
count = 1
while count <= 3:
    print(count)
    count += 1  # essential update
```

for versus while

CONCEPT & REASONING

for is clearer for a known number of values. while is useful when repetition ends because the user supplies a valid input or a stop signal.

EXAMPLE / TAKEAWAY

Known: enter costs for five days → for

Unknown: ask again while amount < 0 → while

Keep asking until input is valid

CONCEPT & REASONING

Use the Unit 3 condition inside a while loop. A negative amount causes a new question; a non-negative amount ends the retry loop.

PYTHON EXAMPLE

```
amount = float(input("Amount: "))
while amount < 0:
    print("Try again")
    amount = float(input("Amount: "))
```

Validation: follow a concrete example

CONCEPT & REASONING

For typed amounts -500, -100 and 2500, the first two are rejected and the third is accepted.

EXAMPLE / TAKEAWAY

-500 → invalid; ask again

-100 → invalid; ask again

2500 → accepted; continue

A sentinel means stop

CONCEPT & REASONING

A sentinel is a special value reserved to end a loop, such as -1. Do not include it in the sum.

PYTHON EXAMPLE

```
amount = float(input("-1 to stop: "))
while amount != -1:
    if amount >= 0:
        total += amount
    amount = float(input("-1 to stop: "))
```

Sentinel example

CONCEPT & REASONING

A sentinel loop can collect an unknown number of amounts. A first input of -1 means there are no ordinary entries.

EXAMPLE / TAKEAWAY

Typed values: 2000, 1500, -1

Sum: 3500

The -1 is not included.

Combine a for loop and a while loop

CONCEPT & REASONING

A fixed-count outer loop can process each day. An inner while loop can retry invalid input for that day without moving to the next day.

PYTHON EXAMPLE

```
for day in range(1, 6):
    expense = float(input("Expense: "))
    while expense < 0:
        expense = float(input("Retry: "))
    total += expense
```

Put an if inside a loop

CONCEPT & REASONING

Conditions can count certain kinds of records. Counting zero-spend days requires one check per entered day.

PYTHON EXAMPLE

```
zero_days = 0
for day in range(1, 6):
    expense = float(input("Expense:
"))
    if expense == 0:
        zero_days += 1
```

Do not reset totals inside loops

CONCEPT & REASONING

Reset accumulators before the first iteration. Otherwise, every new value overwrites the history of earlier values.

PYTHON EXAMPLE

```
total = 0.0    # correct place
for day in range(1, 6):
    amount = float(input("Amount: "))
    total += amount
```

Weekly budget comparison

CONCEPT & REASONING

After recording the five days, compare the total with the allowance. This reuses if / elif / else from Unit 3.

PYTHON EXAMPLE

```
balance = allowance - total
if balance < 0:
    print("Over budget")
elif balance == 0:
    print("Exactly on budget")
else:
    print("Within budget")
```

Worked example: define the state

CONCEPT & REASONING

Prepare the allowance and initialise accumulator/counter variables before starting the daily loop.

PYTHON EXAMPLE

```
allowance = 20000.0
total = 0.0
zero_days = 0
```

Worked example: repeat each day

CONCEPT & REASONING

For five days, gather a valid non-negative amount, add it to total, then count zero values.

PYTHON EXAMPLE

```
for day in range(1, 6):
    amount = float(input(f"Day {day}: "))
    while amount < 0:
        amount = float(input("Try again:
"))
    total += amount
    if amount == 0:
        zero_days += 1
```

Worked example: calculate the report

CONCEPT & REASONING

After the last daily entry, calculate the average and balance and print them with appropriate currency labels.

PYTHON EXAMPLE

```
average = total / 5
balance = allowance - total
print(f"Total: TZS {total:.2f}")
print(f"Average: TZS {average:.2f}")
print("Zero days:", zero_days)
```

Worked example: expected result

CONCEPT & REASONING

For expenses 2000, 3000, 0, 4000, 1000 and allowance 20000, the programme processes all five days.

EXAMPLE / TAKEAWAY

Total: TZS 10000.00

Average: TZS 2000.00

Zero days: 1

Status: Within budget

Five-day test matrix

CONCEPT & REASONING

Plan tests before submission. Cover normal data, all zeros, an exact budget match, over-budget data and a rejected negative input.

EXAMPLE / TAKEAWAY

Example: 1000,2000,3000,4000,5000 → total 15000, average 3000, zero days 0. A negative input must be replaced on the same day.

Off-by-one errors

CONCEPT & REASONING

Python excludes the stop in range(). Incorrect endpoints cause one fewer or one extra iteration.

EXAMPLE / TAKEAWAY

range(1, 5) → 4 days

range(1, 6) → 5 days

range(0, 5) → 5 days, numbered 0–4

Wrong indentation changes behaviour

CONCEPT & REASONING

Inside: performed on every pass. Outside: performed once after the loop. Trace a small example to check the difference.

PYTHON EXAMPLE

```
for day in range(1, 6):
    total += expense
print("Final total", total)  # once
```

A while loop may execute zero times

CONCEPT & REASONING

If the starting condition is False, Python skips the body. This is correct and useful for already-valid input.

PYTHON EXAMPLE

```
amount = 2500
while amount < 0:
    print("Retry")  # skipped
print("Accepted")
```

Optional: break and continue

CONCEPT & REASONING

break exits the nearest loop early. continue skips the remaining body of one iteration. Neither is required for Activity U4.

PYTHON EXAMPLE

```
for n in range(1, 6):
    if n == 3:
        continue
    print(n)  # 1, 2, 4, 5
```

Optional: non-numeric input

CONCEPT & REASONING

A float conversion fails when the user types text such as abc. try / except can handle it, but the core activity only requires handling negative numeric entries.

PYTHON EXAMPLE

```
try:
    amount = float(input("Amount: "))
except ValueError:
    print("Enter a number")
```

Repetition patterns and loop choice

CONCEPT & REASONING

A for loop processes each item from a sequence or iterable. A while loop repeats while its condition is true. Count-controlled repetition has a known count; condition-controlled repetition depends on a changing state; sentinel-controlled repetition ends at a designated value. Collection-based repetition processes each stored observation.

EXAMPLE & RESULT

```
for day in range(1, 6):
    print(day)

remaining = 3
while remaining > 0:
    remaining -= 1
print(remaining)
```

The for loop prints 1 through 5. The while loop ends with remaining equal to 0. A while condition may be false initially.

Nested loops: rows and columns

CONCEPT & REASONING

A nested loop places one loop inside another. For each outer iteration the inner loop executes its full sequence. A 3 by 3 grid therefore has nine coordinate pairs. This pattern supports multiplication tables, spreadsheet cells and image pixels.

EXAMPLE & RESULT

```
for row in range(1, 4):
    for column in range(1, 4):
        print(row, column)
```

(1,1), (1,2), (1,3), (2,1), …, (3,3). Row changes only after the inner loop finishes.

Nested repetition: a multiplication table

CONCEPT & REASONING

The outer loop selects the multiplier and the inner loop selects a second number. Each product is independent, so no running total is needed. Indentation identifies the inner body and separates it from the outer body.

EXAMPLE & RESULT

```
for row in range(1, 4):
    for col in range(1, 4):
        print(row * col, end=" ")
    print()
```

Rows: 1 2 3; 2 4 6; 3 6 9. The print() outside the inner loop ends each row.

Loop state and stopping safely

CONCEPT & REASONING

A loop variable and accumulator retain their most recently assigned values after a non-empty loop. A while loop requires a condition that can become false or an explicit exit. A missing update can create an infinite loop. In a terminal, Ctrl+C commonly interrupts execution; IDLE Shell has Shell > Interrupt Execution. In a notebook, the stop/interrupt control stops the running cell.

EXAMPLE & RESULT

```
count = 0
while count < 3:
    print(count)
    count += 1
print("Final count:", count)
```

0, 1, 2; Final count: 3. Without count += 1, the condition would remain true.

Repeated measurements and meaningful counts

CONCEPT & REASONING

Repeated input can count observations that satisfy a condition, accumulate their total and report a summary. A zero observation is a valid measurement, not necessarily a missing value. A negative rainfall amount is invalid in this fictional example.

EXAMPLE & RESULT

```
total = 0
wet_days = 0
for day in range(1, 6):
    rain = float(input("Rainfall in mm: "))
    while rain < 0:
        rain = float(input("Non-negative mm: "))
    total += rain
    if rain > 0:
        wet_days += 1
print(total, wet_days)
```

For 0, 5, 2, 0, 3 the total is 10 mm and there are three wet days. Non-numeric text still raises ValueError; Unit 6 handles conversion errors.

Activity U4: five-day expenditure tracker

CONCEPT & REASONING

Build a standalone Python programme that records five valid expenses, totals them, computes the average, counts zero-spend days and compares with a weekly allowance.

EXAMPLE / TAKEAWAY

Required: for + while + if; five test cases; readable output with TZS and two decimal places; short notes and code file.

Checklist before submission

CONCEPT & REASONING

Can you explain the loop variable, stopping condition, accumulator, counter, nested validation and test evidence? Can you run the code yourself?

EXAMPLE / TAKEAWAY

Deliverables: StudentID_U4.py and one notes document with algorithm, expected/actual tests and any assistance declaration.

Coming next: lists and functions

CONCEPT & REASONING

Unit 5 will introduce collections of values and named reusable blocks. Loops are already useful without either of them.

EXAMPLE / TAKEAWAY

Readiness: distinguish for/while; predict range values; avoid an infinite loop; trace totals; validate numeric amounts; test loop boundaries.